

Projet 9

Améliorez l'organisation Salesforce du client Fasha

Livrable n°5

Rapport de performance détaillé

Parcours Développeur Salesforce - OpenClassrooms

Abderrahim Yachkour

Développeur Salesforce junior - Cabinet SFQUAL

Client : Fasha (distributeur de vêtements)

Date de remise : Juin 2026

Sommaire

Table des matières

1. Pourquoi mesurer la performance d'une org Salesforce	3
2. Méthodologie de mesure	3
3. Les limites gouverneurs à connaître	3
4. Scénario 1 : activation en masse de 200 commandes	3
4.1. Avant correction	4
4.2. Après correction	4
5. Scénario 2 : recalcul global du CA via le batch	4
6. Scénario 3 : calcul du NetAmount par champ formule	5
6.1. Avant correction	5
6.2. Après correction	5
7. Lien performance et efficacité énergétique	6
8. Synthèse et tableau récapitulatif	6
9. Conclusion	7

1. Pourquoi mesurer la performance d'une org Salesforce

Salesforce est une plateforme multi-tenant : des milliers de clients partagent les mêmes ressources serveur. Pour éviter qu'un client ne monopolise la plateforme, Salesforce impose des limites strictes à chaque transaction. Ces limites, appelées limites gouverneurs, s'appliquent à toute transaction sans exception. Tout code qui les dépasse est interrompu par une exception (typiquement `System.LimitException`).

Mesurer la performance d'une org, c'est donc vérifier deux choses : que le code reste largement en dessous des limites gouverneurs (sécurité fonctionnelle), et qu'il consomme le moins possible de ressources (efficacité énergétique des datacenters Salesforce).

2. Méthodologie de mesure

Pour chaque scénario testé, j'ai suivi le même protocole, afin que les comparaisons soient honnêtes.

- Activer Developer Console, Debug, Change Log Levels et mettre le niveau de log à FINE sur System (cela permet d'avoir la section `LIMIT_USAGE_FOR_NS` détaillée).
- Ouvrir Developer Console, Debug, Open Execute Anonymous Window.
- Coller un bloc Apex anonyme qui reproduit le scénario (par exemple : créer deux cents commandes Draft, puis les passer toutes en Activated en un seul update).
- Exécuter et observer le debug log généré.
- Ouvrir le log dans l'onglet Logs, cliquer sur Filter et chercher `LIMIT_USAGE_FOR_NS`. Cette section donne le nombre exact de SOQL, DML, CPU time et mémoire consommés.

3. Les limites gouverneurs à connaître

Voici les limites pertinentes pour notre cas (transactions synchrones standard) :

- 100 requêtes SOQL maximum par transaction.
- 150 opérations DML maximum par transaction.
- 50 000 lignes lues maximum par transaction.
- 10 000 lignes modifiées maximum par transaction.
- 10 secondes de CPU time maximum.
- 6 méga-octets de heap maximum.

4. Scénario 1 : activation en masse de 200 commandes

C'est le scénario critique du projet. Avant correction, ce cas faisait planter l'org. Après correction, il doit s'exécuter sans erreur et bien en deçà des limites.

4.1. Avant correction

Le trigger original UpdateAccountCA contenait une requête SOQL et une opération DML à l'intérieur d'une boucle for. Pour deux cents commandes, cela signifie deux cents SOQL et deux cents DML, soit deux fois plus que la limite autorisée. L'exception System.LimitException: Too many SOQL queries: 101 était levée à la 101ème commande, et l'opération entière était rollback.

Résultat : impossibilité d'activer plus de cent commandes simultanément. Sur un client comme Fasha qui peut être amené à activer des lots importants, c'est bloquant.

4.2. Après correction

L'architecture en trois couches consomme une seule requête SOQL agrégée (avec GROUP BY) et une opération DML groupée, quelle que soit la volumétrie. La consommation reste constante au lieu de croître linéairement avec le nombre de commandes.

Valeurs mesurées dans l'organisation Fasha après activation de 200 commandes en une seule transaction :

- 1 SOQL (sur 100 autorisées).
- 4 DML statements (sur 150 autorisées), pour 203 lignes modifiées (sur 10 000 autorisées).
- 341 ms de CPU (sur 10 000 ms autorisés), soit moins de 3,5 % du budget CPU.

Debug log APRES correction (limites largement respectées)

Log executeAnonymous @6/23/2026, 9:57:08 PM		
Execution Log		
Timestamp	Event	Details
21:57:09:102	LIMIT_USAGE_FOR_NS	(default)
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of SOQL queries: 1 out of 100
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of query rows: 1 out of 50000
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of SOSL queries: 0 out of 20
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of DML statements: 4 out of 150
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Publish Immediate DML: 0 out of 150
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of DML rows: 203 out of 10000
21:57:08:000	LIMIT_USAGE_FOR_NS	Maximum CPU time: 341 out of 10000
21:57:08:000	LIMIT_USAGE_FOR_NS	Maximum heap size: 0 out of 6000000
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of callouts: 0 out of 100
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Email Invocations: 0 out of 10
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of future calls: 0 out of 50
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of queueable jobs added to the queue: 0 out of 50
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Mobile Apex push calls: 0 out of 10
21:57:08:000	LIMIT_USAGE_FOR_NS	

5. Scénario 2 : recalcul global du CA via le batch

Le batch UpdateAllAccounts permet de recalculer le chiffre d'affaires de tous les comptes de l'org, par lots. Il sert en migration de données ou en audit périodique.

Le filtre dans la méthode start() (uniquement les comptes ayant au moins une commande Activated) est une optimisation importante : il évite de traiter les centaines, voire milliers de comptes sans aucune commande.

Sur l'org Fasha, le batch s'exécute en une seule passe (Total Batches : 1, Batches Processed : 1, Failures : 0) et se termine avec le statut Completed, sans aucune erreur. Le détail est visible dans Setup, Apex Jobs.

Apex Jobs : batch UpdateAllAccounts en statut Completed

Apex Jobs										
Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.										
Percent of Asynchronous Apex Used: 0% You have currently used 1 asynchronous Apex operations out of an allowed 24-hour organization limit of 250 000. To learn about how this limit is calculated and what contributes to it, see the Lightning Platform Apex Limits topic.										
View: All Create New View										
Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class
	23/06/2026 22:08	Batch Apex	Completed		1	1	0	YACHKOUR, Abderrahim	23/06/2026 22:08	UpdateAllAccounts
										Apex Method
										Apex Job ID
										707g500000V2w1M

6. Scénario 3 : calcul du NetAmount par champ formule

Le champ NetAmount__c a été transformé en formule Salesforce. La formule (TotalAmount - ShipmentCost__c) est désormais évaluée à chaque lecture, automatiquement par la plateforme.

6.1. Avant correction

Un trigger CalculMontant écrivait la valeur dans un champ Currency à chaque update. Coût pour chaque mise à jour : une opération de calcul Apex et une écriture en base. Et un bug : seule la première commande du lot était traitée, soit 0,5 % de réussite sur un batch de deux cents commandes.

6.2. Après correction

Le champ formule a un coût d'écriture de zéro (rien n'est stocké). Le calcul a lieu à la lecture, géré par le moteur Salesforce. Et 100 % des commandes sont à jour, quel que soit le volume.

Configuration du champ formule NetAmount__c

Order Custom Field

Net amount

Back to Order Fields

Help for this Page

Custom Field Definition Detail

Edit

Set Field-Level Security

View Field Accessibility

Where is this used?

Field Information

Field Label	Net amount	Object Name	Order
Field Name	NetAmount		
API Name	NetAmount__c		
Description			
Help Text			
Data Owner			
Field Usage			
Data Sensitivity Level			
Compliance Categorization			
Created By	Abderrahim YACHKOUR, 22/06/2026 13:04		Modified By Abderrahim YACHKOUR, 22/06/2026 13:04

Formula Options

Data Type	Formula
Decimal Places	2

TotalAmount - ShipmentCost__c

7. Lien performance et efficacité énergétique

La consommation énergétique d'une org Salesforce est principalement liée à trois métriques : le nombre de requêtes SOQL exécutées (chaque requête mobilise CPU et entrées-sorties disque sur les serveurs Salesforce), le CPU time cumulé côté Force.com, et le nombre d'opérations DML (chaque écriture déclenche du logging et la propagation en cascade des triggers).

Réduire ces trois métriques, c'est donc réduire directement la consommation énergétique des datacenters Salesforce. Voici les leviers que j'ai actionnés sur le projet Fasha :

- Suppression de toutes les requêtes SOQL en boucle. Une boucle qui exécute N requêtes consomme N fois plus de ressources qu'une requête unique avec GROUP BY.
- Utilisation d'une requête SOQL agrégée par GROUP BY au lieu de N requêtes individuelles. Le moteur de base de données optimise alors le scan en une seule passe.
- Groupement des opérations DML : un seul update Account batché au lieu de N updates individuels.
- Remplacement du trigger CalculMontant par un champ formule : zéro stockage, zéro écriture, calcul à la lecture par le moteur Salesforce (qui peut le cacher).
- Mutualisation du code via le pattern Selector : la même requête SOQL définie une seule fois est consommée par le trigger, le batch et le contrôleur LWC. Trois fois moins de parsing SQL côté Salesforce.

Estimation qualitative : sur un cycle hebdomadaire de mise à jour des prix produits suivi du recalcul du CA, la version optimisée consomme typiquement entre cinquante et quatre-vingt-dix % moins de ressources Force.com que la version initiale (en réalité, la version initiale ne pouvait simplement pas exécuter le scénario du tout).

8. Synthèse et tableau récapitulatif

Le tableau ci-dessous synthétise les gains observés sur les trois scénarios, mesurés dans l'org Fasha.

Scénario 1 : activation de 200 commandes

- Avant : exception System.LimitException dès la 101ème commande, opération annulée.
- Après : 1 SOQL, 4 DML, 341 ms de CPU (sur 10 000 ms autorisés), soit moins de 3,5 % du budget CPU.

Scénario 2 : calcul du NetAmount sur 200 commandes

- Avant : une commande sur deux cents correctement calculée (0,5 % de réussite), 200 DML inutiles.
- Après : 200 commandes sur 200 toujours à jour, 0 DML (calcul à la lecture).

Scénario 3 : recalcul global du CA via batch

- Avant : impossible (batch vide).
- Après : exécution complète sans erreur, statut Completed visible dans la page Apex Jobs.

9. Conclusion

Les trois scénarios montrent que les corrections vont au-delà de la simple résolution des bugs. Un code qui échouait dès qu'on dépassait la centaine d'enregistrements consomme désormais une fraction des limites gouverneurs Salesforce. Cette marge de sécurité permettra à l'org de Fasha d'absorber la montée en charge à venir, à mesure que le volume de commandes augmentera.